

Dimension

Deck

Jesús José Espinoza Torruco	-	A01781963
Gonzalo Zamarron Orrantia	-	A01782739
Vladimir Piñera Reyes	-	A01786772

Elevator Pitch

Dimension Deck is a retro top-down action roguelite built entirely in vanilla JavaScript where you fight through procedurally generated rooms using a deck of cards.

Every room you clear can hide a new card, a hidden chest, or a boss encounter. You build your deck as you explore two distinct dimensions, the Dark Ages and Old West, and the hand you've assembled determines whether you survive what's behind the next door.

What makes it different: cards don't just execute when you press a button. Half your deck fires automatically. You're managing real-time movement and card synergies at the same time.

The closest comparison is The Binding of Isaac meets Slay the Spire, but with a twist nobody else has tried: automated trigger cards that run without any player input, turning your deck into a living system rather than a set of tools.

Game Strategy

We decided to build a top-down roguelite with real-time card-based combat. The core idea is that the player navigates a procedurally generated map, collects cards along the way, and uses them to survive encounters without ever pausing the game.

We chose this approach because it let us apply every topic from the course at once: object-oriented programming, data structures, relational databases, and APIs. We also wanted a real technical challenge, not just another CRUD app.

Our main references were The Binding of Isaac and Slay the Spire, but our take is different: automatic cards fire without any player input, which forces the player to think of their deck as a passive system running in parallel to their movement.

The methodology we used was SCRUM, with 5 weekly sprints, each with concrete deliverables defined from the start. Three developers, GitHub as our shared repository.

SCRUM

Methodology

We ran the project across 5 weekly sprints.

- Sprint 1 covered the engine core
- Sprint 2 added gameplay and the card system
- Sprint 3 built the procedural map
- Sprint 4 was all UI/UX
- Sprint 5 was testing and delivery

Every week we held a Scrum Master review where the current Scrum Master led the session walking through what was completed, identifying blockers, and reprioritizing the backlog for the next sprint. We rotated the Scrum Master role each sprint so every team member had to take ownership of the process and understand the full picture of the project, not just their own tasks.

This rotation turned out to be one of the most valuable parts of our workflow. When it was your week to run the review, you had to know what everyone else was building, which pushed better communication across the team.

Database

Design and Integration

The backend operates on Node.js and Express, using a MySQL database to process real-time game data via an API. To ensure reliability, all critical operations are handled by stored procedures: `sp_start_run` initializes games, `sp_end_run` calculates final scores based on victories, rooms cleared, enemies defeated, and damage dealt, while other procedures manage card upgrades, the shop economy, and activity logging.

To maintain data integrity, the system utilizes automated triggers that work in the background. These triggers handle essential housekeeping, such as initializing player states upon startup, preventing invalid score changes, blocking access to completed runs, tracking card acquisition, and ensuring the game's credit economy never drops below zero. This combination of procedures and triggers creates a robust, self-regulating environment that keeps the game data consistent and secure.

Views and Analytics

To make analytics more accessible, we use a series of SQL views that provide pre-formatted data without needing to access raw tables directly. These views power the frontend admin panel and leaderboards, centralizing the logic within the database rather than spreading it across multiple API routes. For instance, `v_user_run_summary` and `v_run_detail` provide history and event timelines, while `v_global_stats`, `v_top_cards`, and `v_card_win_rate` offer deeper insights into player behavior and game balance. We even use a view, `v_shop_catalog`, to manage the store; by filtering cards with a cost greater than zero, any new item added to the master list automatically appears in the shop, requiring no further code changes.

Frontend Website

Beyond the game itself, we built a companion website served from the same Node.js backend. It consists of five separate HTML pages, each connected to the same REST API the game uses.

The Login page handles user registration and authentication, feeding into the session that links run data to a specific player. The Tutorial page hosts a walkthrough video explaining the card system, dimensions, and controls for new players. The Cards Catalog page pulls from the `v_shop_catalog` view and displays every card in the game with its stats, rarity, and effect description — useful for planning a run strategy before playing.

The Stats Dashboard reads from `v_user_run_summary`, `v_card_usage`, and `v_card_win_rate` to show each player their personal history: runs completed, best score, most used cards, and win rate by deck composition. The Admin Panel gives authorized users access to global stats via `v_global_stats` and `v_top_cards`, and lets them inspect any active run through `v_active_runs` and `v_run_detail`.

The website shares the same dark pixel-art color palette as the game, keeping the visual identity consistent across both surfaces.

Challenges and How We Solved Them

Merge conflicts: Working on parallel branches (cards, rooms, enemies) caused constant conflicts when merging. We fixed this by establishing a mandatory PR workflow, nobody merges without team review, and by splitting modules with clear interfaces to reduce collisions.

Sprites: Spritesheets were not rendering correctly. Offsets were miscalculated, animations were out of sync, and sizes were inconsistent across characters. We built a tool to visualize individual frames and standardized all sizes in a constants file.

Lack of communication: There were sprints where everyone worked in isolation and we ended up duplicating code or building incompatible functions. We addressed this with short daily check-ins, even if just a quick message, and used the GitHub Projects board so everyone could see what the others were working on.

Main Lessons Learned

Modularity matters from day one. Designing the base classes (Entity, Card, Room) well from the start saved us weeks of refactoring later.

SCRUM works even for small teams. Weekly sprints with defined deliverables gave us structure and kept the project moving in a clear direction.

Constant communication beats perfect communication. A short daily update prevents technical blockers and misunderstandings before they become real problems.

Early testing saves the project. Bugs we caught in Sprint 2 would have been critical if we had discovered them in Sprint 5.

Documenting design decisions (the GDD) was the best time investment we made. Whenever there was ambiguity about how something should work, the document aligned all three of us.

Reflection

Jesus Espinoza

This project showed me that architecture matters more than features. When we designed the base classes the rest of the game grew naturally without having to rewrite anything from scratch.

Working with SCRUM taught me to commit to a realistic scope. Several times I wanted to add new mechanics to the game, but the sprint had clear objectives and that forced me to prioritize what actually mattered.

The merge conflict problem was frustrating at first, but it made me actually learn Git: feature branches, PRs, rebase. I can no longer imagine working on a team project any other way.

My conclusion is that a video game is a system of systems. Each module needs to work independently and communicate with the rest through well-defined interfaces. That applies equally to any software project.

Reflection

Vladimir Piñera

Implementing the active and automatic card system was the biggest technical challenge I faced. I had to think in specific triggers that the game could require at certain moments of combat.

Enemy AI surprised me. What seems simple hides a lot of difficulties: collisions between enemies, behavior in corners, knockback. Each archetype ended up needing its own fine-tuning.

The lack of communication on the team affected me directly in Sprint 3. I spent two days working on something that already existed. After that I made a point of sharing my progress more often, even when it was small.

My conclusion is that coding as a team is a different skill from coding alone. It requires empathy, clarity in your code, and willingness to refactor your own work when the team needs it.

Reflection

Gonzalo Zamarrón

Fixing the sprite problem was an entire school in itself. I learned to read spritesheets, calculate frame offsets, sync animations with character states, and debug visually in Canvas 2D — things no tutorial really teaches you.

Designing the procedural map introduced me to graph algorithms in a practical way. The generator creates 20-30 nodes and uses depth-first search to fix the deepest node as the boss room. Understanding that required studying outside of class.

As the person responsible for UI, I understood that the player experience is just as important as the game logic. A badly placed cooldown indicator or an unreadable font can ruin a run that would otherwise be perfect.

My final reflection is that design and code are not separate. Every visual decision has technical implications and every technical decision has visual ones. The best moments of the project were when those two things aligned.